

Efficient In-Context Learning for 3D Medical Image Segmentation

Submitted to the
Albert-Ludwigs-Universität Freiburg

by Tidiane Camaret Ndir



Master's Thesis

Efficient In-Context Learning for 3D Medical Image Segmentation

Submitted on **September 23, 2026**
to the **Albert-Ludwigs-Universität Freiburg**
by **Tidiane Camaret Ndir**

Examined by

Division of Medical Physics, Department of Radiology

Medical Center – University of Freiburg

Albert-Ludwigs-Universität Freiburg

Enter
your
student
ID.

Add
your
ORCID
here (or
remove
\\or-
cidlink
if you
don't
have
one) –
\\orcidlink{y
orcid}.

Examiner
name.

Enter your data int the Declaration.

Declaration

I hereby declare, that I am the sole author and composer of my thesis and that no other sources or learning aids, other than those listed, have been used.

Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.

I also hereby declare that my thesis has not been prepared for another examination or assignment, either in its entirety or excerpts thereof.

Freiburg, September 23, 2026

Place, date

Tidiane Camaret Ndir

Acknowledgements

Further thanks go to the corporate design team at the Albert-Ludwigs-Universität Freiburg and Leander Marius Bürkin  for this L^AT_EX-template.

Thanks
to your
supervi-
sor(s),
e.g.
Marco
Reisert
and
Robin T.
Schirrmeis-
ter.

Abstract

Zusammenfassung

Contents

1	Motivation	1
2	Fundamentals	2
2.1	In-Context Segmentation	2
2.2	Volumetric Encoders and Attention	2
2.3	Synthetic Training Data	2
2.4	Related Work	2
3	Methodology	3
3.1	Problem Setup and Backbone	3
3.2	Bi-Axial Image–Label Fusion	3
3.3	Coarse-to-Fine Cascade	4
3.4	Synthetic Task Generation	6
3.5	Training Objective	7
4	Results	8
4.1	Experimental Setup	8
4.2	Image–Label Fusion	8
4.3	Coarse-to-Fine Cascade	8
4.4	Synthetic Task Generation	8
5	Discussion	9
6	Conclusion	10
6.1	Tables and References	11
6.2	Auto-complete in VSCode	11
6.3	”, Quotations and Code	12
6.4	Clipboard Manager, Git and more	13
6.5	Glossary	13
	Symbols	I
	List of Figures	II
	List of Tables	III
	List of Algorithms	IV
	Glossary	V
	References	VI

1 Motivation

In-context learning (ICL) recasts segmentation as few-shot matching: given a target scan and a small set of (image, mask) context pairs illustrating the structure of interest, a model predicts a mask for the target with no task-specific fine-tuning. This removes the need to retrain for every new anatomical class – valuable for structures that are rare, newly defined, or annotated at only a handful of sites.

Most ICL segmentation models were designed and validated in 2D. Moving to native 3D volumes is attractive – CT/MRI are inherently volumetric, and slice-wise processing discards through-plane context needed to resolve thin or elongated structures – but it sharpens an already difficult resolution/compute trade-off: dense cross-attention between a target volume and its context volumes scales with the cube of side length, so a design affordable at 64^3 becomes impractical at 256^3 .

This trade-off is not one problem but three, and this thesis addresses each with its own mechanism:

1. **How should image and label evidence be combined?** Concatenating image and mask channels before a shared encoder, or fusing them once through a single operation, commits to one fusion point before most of the network’s capacity is applied. section 3 proposes *bi-axial image–label attention*, keeping the two streams separate and letting them exchange information repeatedly.
2. **Where should compute be spent?** Re-scanning an entire volume with a sliding window at every processing step wastes compute on regions already known not to contain the structure of interest. section 3 proposes a *coarse-to-fine cascade* that restricts fine processing to the region a coarse pass has already localized, connected across levels by a query prior and register tokens that carry a gradient path between resolutions.
3. **Where does training signal come from beyond limited annotation?** Real (image, mask) pairs are limited to the classes and appearance distribution covered by the training cohort. section 3 proposes a *synthetic task generator* that repaints real anatomical geometry with novel, controllable appearance, producing label-perfect training tasks beyond the annotated cohort.

Contributions.

- A bi-axial image–label attention mechanism for in-context 3D segmentation, contrasted against early-concatenation (Medverse [3]) and one-shot fusion (Iris [2]) baselines.
- A coarse-to-fine cascade in which the query prior, region-restricted fine processing, and cross-level register training are evaluated against the sliding-window-over-the-whole-volume design used by prior work.
- A GMM-based synthetic task generator with a texture-noise variant that better matches real tissue’s spatial autocorrelation, evaluated against real-only and i.i.d.-noise training.
- Evaluation on TotalSegmentator CT and a range of out-of-distribution cohorts and modalities, reporting where each mechanism helps and where it does not.

Outline. section 2 introduces in-context segmentation, the volumetric encoder and attention machinery the method builds on, and synthetic-data background, then positions UniverSeg, Iris, and Medverse against the three axes above. section 3 details the model. section 4 evaluates each axis in turn. section 5 synthesizes what the evidence supports and what remains open, and section 6 summarizes.

2 Fundamentals

2.1 In-Context Segmentation

Given a target image I^t and a context set $\{(I_k^c, L_k^c)\}_{k=1}^K$ of image/mask pairs for the same class, in-context segmentation predicts a mask $\hat{L}^t$ for the target in a single forward pass, with no gradient update and no class-specific parameters. This sits between two more familiar paradigms: closed-set supervised segmentation, which fixes a label set at training time and must be retrained (or fine-tuned) to add a class; and interactive segmentation, where a human supplies clicks or boxes on the target image itself, per image, at inference time. In-context segmentation instead supplies the “prompt” as a handful of *other*, already-labeled examples of the same class – closer to how a radiologist might be shown a few example cases of a rare finding and asked to find it in a new scan. This is valuable in medical imaging specifically because annotation requires domain expertise, new structures and pathologies are defined faster than segmentation datasets can be built for them, and per-site protocol differences limit how far a closed-set model trained elsewhere transfers.

2.2 Volumetric Encoders and Attention

Convolutional volumetric backbones. 3D U-Net [?] and its self-configuring descendant nnU-Net [4] encode a volume through a downsampling encoder into a feature pyramid, then decode back to full resolution via a mirrored upsampling path with skip connections carrying fine detail past the bottleneck. Deeper encoder stages have coarser spatial extent and more channels; shallower stages retain spatial detail at low channel count. This split – a coarse, semantically rich bottleneck feeding a computationally light attention stack, and fine, spatially precise early stages reserved for the decoder – is exactly what the coarse/fine tap split in subsection 3.1 reuses. UNETR [?] is a prior example of combining a convolutional decoder with transformer attention for 3D segmentation.

Attention. Scaled dot-product attention [?] computes, for each query token, a weighted average of value vectors, with weights given by the similarity between the query and each key:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V. \quad (1)$$

Multi-head attention runs several such operations in parallel subspaces and concatenates the results, letting different heads attend to different relationships in the same input.

Position encoding. Attention itself is permutation-invariant, so position must be injected separately. Rotary position embedding (RoPE) [?] does this by rotating each query/key vector by an angle proportional to its position before the dot product, so the result depends only on the *relative* position between the two tokens rather than their absolute indices. In 3D, this is applied axially: the head dimension is split into three chunks, one per spatial axis, each rotated by an angle proportional to that axis’s physical coordinate (scaled by voxel spacing) – letting attention between two cells at different cascade resolutions still reflect their true physical distance, not just their grid index.

2.3 Synthetic Training Data

Domain randomization trains a model on procedurally varied synthetic data so it does not overfit to the narrow appearance distribution of a single acquisition protocol or training cohort. SynthSeg [?] demonstrated this at scale for medical segmentation: training pairs are generated by sampling a random Gaussian-mixture appearance conditioned only on a label map, so the resulting segmentation network never sees a real image of any specific contrast during training yet generalizes across MRI contrasts and resolutions. GIN/IPA [?] takes a complementary, appearance-only approach to single-source domain generalization: a random nonlinear intensity warp (GIN), optionally blended across spatially distinct regions of the same volume (IPA) to break spurious global appearance correlations, achieving strong generalization from randomizing appearance alone, without touching shape or geometry. Both results motivate treating appearance randomization as a distinct lever from geometric augmentation – a question the synthetic task generator (subsection 3.4) revisits directly by ablating *how* painted appearance is randomized (i.i.d. vs. spatially correlated texture noise) while holding geometry fixed.

2.4 Related Work

Three prior systems attempt in-context medical image segmentation, each making a different choice on the three axes this thesis organizes around.

UniverSeg [?] popularized the approach with a CrossBlock architecture that densely cross-attends target and context features at full resolution – no coarse-to-fine structure, and no synthetic-task training.

Iris [2] encodes each context pair into a compact per-class task embedding and fuses it into the target through a pixel shuffle-unshuffle operation – a single fusion step, applied once, with no iterative refinement across scales.

Table 1: Positioning against prior in-context segmentation models on the three axes this thesis organizes around.

	Fusion	Coarse-to-fine	Synthetic tasks
UniverSeg	dense cross-attn.	no	no
Iris	one-shot pixel shuffle	no	no
Medverse	early concatenation	full-volume re-scan	no
Ours	bi-axial (repeated)	region-restricted, register-carried	GMM repaint + texture

Medverse [3] performs next-scale autoregressive segmentation, inspired by next-scale image generation [?]: at each step, the target and the previous step’s (perturbed) prediction are concatenated and processed by a shared U-Net, together with the real context pairs. Every step still re-scans the *entire* volume with a sliding window – only the input changes with scale, not the spatial extent processed.

Deep Neural Patchworks [?] stacks levels of increasing resolution for large-volume segmentation, but targets single-model supervised segmentation rather than in-context, few-shot prediction – an architectural precedent for coarse-to-fine cascades outside the ICL setting.

3 Methodology

3.1 Problem Setup and Backbone

Given a target volume I^t and a context set $\{(I_k^c, L_k^c)\}_{k=1}^K$ of image/mask pairs for the same anatomical class as the (unknown) target mask, the model predicts a target mask $\hat{L}^t$. Prediction proceeds through a coarse-to-fine cascade of M resolution levels with decreasing voxel spacing $r_1 > r_2 > \dots > r_M$; each level produces a full mask that seeds the next, finer level (subsection 3.3). Figure 1 contrasts our architecture with Medverse’s.

Shared encoder. All $T = K + 1$ volumes of a task – the K context images and the target – are passed once through a single shared convolutional encoder (a from-scratch, nnU-Net-style plain-convolution stack [4], without external pretraining), producing a feature pyramid per volume. Two disjoint subsets of its stages are read out: *coarse* stages, resampled to a common $R \times R \times R$ token grid ($N = R^3$ cells per volume) and consumed by the attention stack below; and *fine* stages, kept at native resolution and reserved for the decoder’s skip connections (subsection 3.2). This split keeps the attention stack on a small, fixed-size token grid while still allowing a full-resolution decoded mask.

Tokenization. Each cell of every volume yields two tokens sharing one row – an image token z^{img} and a label token z^{lbl} – assembled into a single sequence together with a small number of learned *thinking rows*, shared across all volumes, that give the attention stack fixed-size read/write capacity not tied to any single spatial cell. Context rows always carry the ground-truth mask in their label token; the target’s label token instead starts from either a neutral placeholder or the previous cascade level’s prediction (subsection 3.3), since the target mask is exactly what is being predicted.

3.2 Bi-Axial Image–Label Fusion

How a model combines image and label evidence is a central design choice. Medverse [3] concatenates image and label channels before a shared U-Net, merging the two streams before most of the network’s capacity is applied. Iris [2] instead fuses image and label features once, through a pixel shuffle–unshuffle operation between context and target. Both commit to a single fusion point. We instead keep image tokens z^{img} and label tokens z^{lbl} (subsection 3.1) as separate streams throughout the network and let them exchange information repeatedly, over two attention axes applied at every layer:

Draft diagram, ported from the conference draft; finalize labels/scale once the architecture is settled.

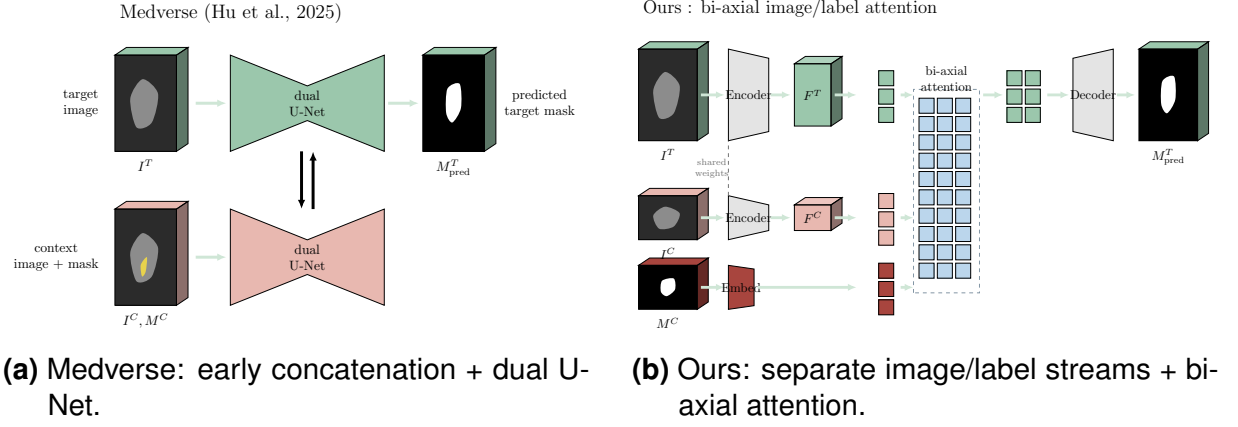


Figure 1: Architecture overview, contrasted against Medverse’s early-concatenation fusion.

- **Slot axis:** at each cell, the image and label token attend to each other, letting label evidence sharpen the image representation and vice versa, independently per cell.
- **Row axis:** every cell of every volume – context, target, and the thinking rows – attends over the entire token sequence at once, using 3D axial rotary position embeddings so attention is a function of physical distance between cells rather than raw grid index. This single joint attention performs within-volume self-attention and cross-context matching together, since tokens from every volume already share one sequence.

Figure 2 illustrates both axes within a single layer.

Only the target’s post-attention *image* token is read out for decoding; the label token is discarded after the last layer, since it started from a placeholder rather than real image evidence for the target row. The decoded image tokens are projected back to the coarse token grid and progressively fused with the encoder’s fine-resolution skip features (subsection 3.1) via upsampling and convolution, one step per fine stage, into the final full-resolution mask logits.

3.3 Coarse-to-Fine Cascade

The cascade connects consecutive resolution levels through three mechanisms, each replacing a coarser design choice made by Medverse or Iris.

Query prior. Medverse [3] feeds an upsampled, perturbed ground-truth mask into an extra U-Net branch at every step; Iris [2] uses no prior at all. At level $\ell > 1$, we instead

Draft diagram, ported from the conference draft; finalize once the layer factorization is settled.

Bi-Axial Attention (one layer)

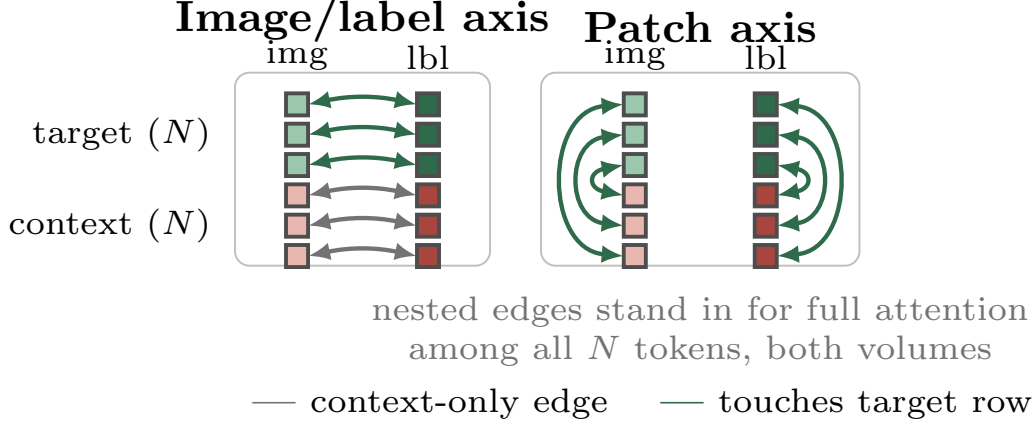


Figure 2: Bi-axial attention within one transformer layer, shown for a single context volume ($K=1$ for clarity) and 3 of its N spatial tokens per stream. Left: the slot axis – image and label tokens attend to each other within the same cell. Right: the row axis – tokens attend across cells and, for the target, across contexts. Green edges touch the target row, which cross-attends read-only into context and whose label-stream input is the query prior (subsection 3.3) rather than ground truth.

initialize the target's label token directly from level $\ell-1$'s own prediction:

$$z_{\ell,0}^{\text{lbl}} = \text{LblEmbed}(\text{Warp}_{\ell-1 \rightarrow \ell}(\hat{L}_{\ell-1}^t)), \quad (2)$$

where Warp resamples level $\ell-1$'s prediction onto level ℓ 's crop grid, correcting for the two levels' different centers, field of view, and any augmentation applied independently to each. This makes the previous level's belief part of the same token stream that attends bi-axially with the image (subsection 3.2), rather than a separate side channel. Figure 3 shows this flow across two levels.

Region-restricted fine processing. Medverse and Iris re-scan the entire volume with a sliding window at every step. Once a coarse level has localized the structure of interest, we instead restrict the sliding window at the next, finer level to the coarse prediction's foreground region, thresholded and padded by a fixed margin. Compute at fine levels then scales with the size of the structure rather than the size of the input volume, at the cost of depending on the coarse level's localization being approximately correct.

Draft diagram, ported from the conference draft; finalize once the number of levels M is settled.

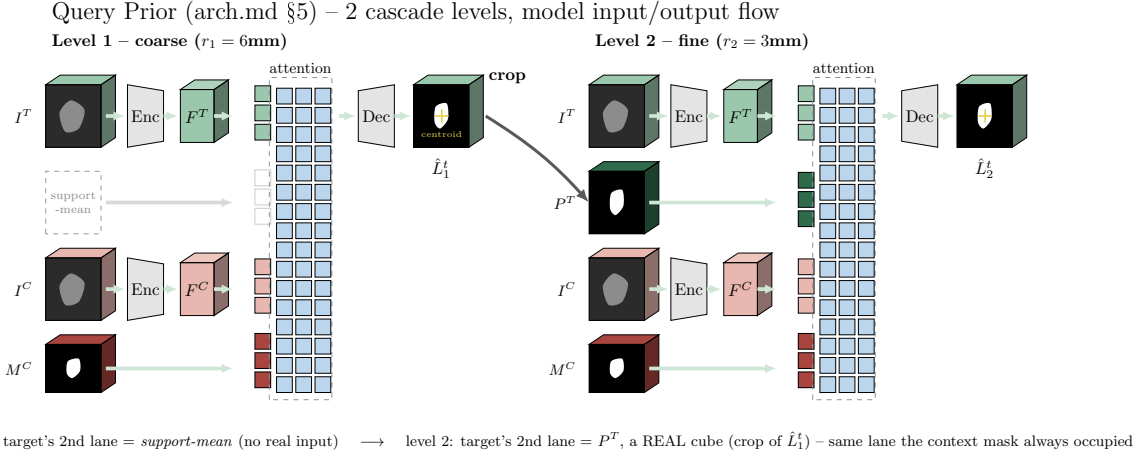


Figure 3: Query image prior across two cascade levels, with the full model input/output flow. At level 1 the target has no real prior yet (dashed support-mean placeholder); at level 2 the target's second lane is a real crop of level 1's own prediction $\hat{L}_1^t$ – the same lane the context mask always occupies.

Cross-level training via register tokens. Medverse and Iris connect levels only through the rendered prediction feeding the next step, with no gradient path between them. We add a small set of learned register tokens $\mathbf{r}_\ell$ that participate in level ℓ 's attention alongside the image and label tokens; their final state is carried into level $\ell+1$ via a learned projection and a level-independent type embedding:

$$\mathbf{r}_{\ell+1,0} = W_r \mathbf{r}_{\ell,\text{final}} + \mathbf{e}_{\ell+1}. \quad (3)$$

Unlike the query prior, this path is not detached between levels: level $\ell+1$'s loss can backpropagate into level ℓ 's own attention, giving the model a direct incentive to carry forward whatever helps the next level – e.g. uncertainty or cues about neighboring structures – beyond what the rendered mask alone can encode.

3.4 Synthetic Task Generation

Real (image, mask) pairs are limited to the anatomical classes and appearance distribution covered by the training cohort. To broaden training beyond this, a fraction p_{synth} of training tasks are drawn from a synthetic generator instead of real context/target subjects.

Supervoxel repainting. A real-anatomy volume is first oversegmented into supervoxels (unsupervised, appearance-agnostic regions). One supervoxel is treated as the target class, and its binary mask is label-perfect by construction. Each of the $K+1$ volumes needed for a task is then produced by repainting the same supervoxel geometry with independently sampled per-class Gaussian intensities (a Gaussian mixture over

classes), giving $K+1$ (image, mask) pairs that share identical anatomy but differ in appearance – no real cross-subject registration or annotation is required.

Paint noise. The default painting noise is i.i.d. per voxel: it matches each class’s target intensity variance but has near-zero spatial autocorrelation, unlike real CT/MRI tissue (measured ≈ 0.6 autocorrelation at native resolution, versus ≈ 0.0 for i.i.d. paint). An optional multi-octave variant instead sums several coarse-to-fine random fields before painting, producing spatially correlated texture that better matches real tissue statistics while leaving the per-class intensity variance unchanged.

3.5 Training Objective

Each cascade level is supervised independently at its own resolution with a combined binary cross-entropy and soft Dice loss, summed over the M levels:

$$\mathcal{L} = \sum_{\ell=1}^M \lambda_{\ell} (\mathcal{L}_{\text{BCE}}^{\ell} + \mathcal{L}_{\text{Dice}}^{\ell}) . \quad (4)$$

Level weights λ_{ℓ} are uniform ($\lambda_{\ell}=1$) in the reference configuration; the query-prior path is detached between levels (subsection 3.3), so each level’s loss trains its own prediction without back-propagating through the coarser level’s mask.

4 Results

4.1 Experimental Setup

4.2 Image–Label Fusion

4.3 Coarse-to-Fine Cascade

4.4 Synthetic Task Generation

5 Discussion

6 Conclusion

ToDoS

Select the correct language in the first rows of `main.tex`. The % makes a line a comment.

Generate a PDF (Visual Studio Code: click on the green arrow in the top right corner).

Check the PDF (Visual Studio Code: click on the two rectangles with the magnifying glass in the top right corner).

Talk about this template with your supervisor: Should you add a declaration for the use of AI? Currently no official AI-declaration is given. You can find one in the pull requests of the github repository.

With CTRL pressed you can click on the PDF and jump to the corresponding place in your `.tex` files.

Check the Errors. (Visual Studio Code: bottom left) There should be one warning because the first ToDo pushes down the second one: LaTeX Marginpar on page 2 moved.

In the `main.tex` you can press ALT + SHIFT and the ARROW UP and DOWN keys to create a multiline cursor, comment out all the sections you are currently not working on and remove the multiline cursor with ESC.

Keep the following examples and todos till the end and check them out sometimes.

Adjust your License Notes to reflect the used programs.

For Printing: Remove the `todonotes`-package in `setup.tex` to find all Todos.

Table 2: A nice table with a nice caption.

Value & Unit	Value	Unit
9.82 m s ⁻²	9.82	m s ⁻²

For Printing: Choose the correct formatting (search `print` in `setup.tex`).

You can remove the glossary and the symbolslist to make it easier, especially in a Bachelor's Thesis.

6.1 Tables and References

Table 1

This is a cited book [1].

Store and display plots and other graphs as `.pdf` files.

Include Algorithm 1 in the same way you include equations like Equation 4:

$$\begin{aligned}y &= 3x^2 + 5x^2 + 4x + 4x + 7 + 1 \\&= 8x^2 + 8x + 8 \\y &= 8 \cdot (x^2 + x + 1)\end{aligned}\tag{5}$$

6.2 Auto-complete in VSCode

There are also some auto-complete features: type `@` + letter to get the letters greek version.

Type `@v` + letter for the alternative form (like φ for ϕ).

You can also use shortcuts for environments like

$$BAL\tag{6}$$

Type BAL and select the correct option from the autocomplete. Maybe you have to try various ones. The B is the base command and AL is the abbreviation for align.

If you want an environment without an index, you can use BS as base command.

Same for changes to your font, like bold or italic: FIT gives *text*

6.3 ", Quotations and Code

Use " or file-names, file_names and code_snippets.

Algorithm 1: The Colors from the Corporate Design of the University of Freiburg. The keywords are bold and the comments blue. This can be changed in setup.tex.

```
from dataclasses import dataclass
```

```
# https://cd.uni-freiburg.de/farben/
```

```
@dataclass
```

```
class MainColors:
```

```
    blue = "#344A9A"
```

```
    black = "#000000"
```

```
    white = "#FFFFFF"
```

```
@dataclass
```

```
class BackgroundColors:
```

```
    blue_100_percent = "#344A9A"
```

```
    blue_80_percent = "#5D6BAD"
```

```
    blue_60_percent = "#868DC2"
```

```
    blue_80_percent = "#AFB1D8"
```

```
    blue_20_percent = "#D7D8EC"
```

```
    dark_blue = "#00004a"
```

```
    sand_100_percent = "#f6f1e3"
```

```
    sand_50_percent = "#faf8f1"
```

```
@dataclass
```

```
class AdditionalColors:
```

```
    green_100_percent = "#00a082"
```

```
    green_80_percent = "#27b29b"
```

```
    green_60_percent = "#7bc6b4"
```

```
green_80_percent = "#afdace"  
green_20_percent = "#daede7"  
brown_100_percent = "#8f6b30"  
brown_80_percent = "#a58959"  
brown_60_percent = "#bca683"  
brown_80_percent = "#d2c4ac"  
brown_20_percent = "#e9e1d6"  
yellow_100_percent = "#ffe863"  
yellow_80_percent = "#ffed82"  
yellow_60_percent = "#fff1a1"  
yellow_80_percent = "#fff6c1"  
yellow_20_percent = "#fffae0"  
pink_100_percent = "#f5c2ed"  
pink_80_percent = "#f7cef1"  
pink_60_percent = "#f9daf4"  
pink_80_percent = "#fbe7f8"  
pink_20_percent = "#fdf3fb"
```

6.4 Clipboard Manager, Git and more

Clipboard Manager help with copying multiple different things at once. On Windows you can press WINDOWS + V to activate the history and choose older things you copied.

Use Git ([click here](#)).

6.5 Glossary

Click them, the page numbers in the glossaries are also clickable.

name one

name one

name one

long form of name two (short2)

short2

short2

a symbol

a symbol

a symbol

Big Letter at start: Name one

plural of name one

name one

Big Letter at start: Short2

short2

short2s

Big Letter at start: *A symbol*

a symbol

Symbols

This document is incomplete. The external file associated with the glossary ‘symbols’ (which should be called `main.sls`) hasn’t been created.

Check the contents of the file `main.slo`. If it’s empty, that means you haven’t indexed any of your entries in this glossary (using commands like `\gls` or `\glsadd`) so this list can’t be generated. If the file isn’t empty, the document build process hasn’t been completed.

Try one of the following:

- Add `automake` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[automake]{glossaries-extra}
```

- Run the external (Lua) application:

```
makeglossaries-lite.lua "main"
```

- Run the external (Perl) application:

```
makeglossaries "main"
```

Then rerun $\text{\LaTeX}$ on this document.

This message will be removed once the problem has been fixed.

List of Figures

1	Architecture overview, contrasted against Medverse’s early-concatenation fusion.	4
2	Bi-axial attention within one transformer layer, shown for a single context volume ($K=1$ for clarity) and 3 of its N spatial tokens per stream. Left: the slot axis – image and label tokens attend to each other within the same cell. Right: the row axis – tokens attend across cells and, for the target, across contexts. Green edges touch the target row, which cross-attends read-only into context and whose label-stream input is the query prior (subsection 3.3) rather than ground truth.	5
3	Query image prior across two cascade levels, with the full model input/output flow. At level 1 the target has no real prior yet (dashed support-mean placeholder); at level 2 the target’s second lane is a real crop of level 1’s own prediction $\hat{L}_1^t$ – the same lane the context mask always occupies.	6

List of Tables

1	A nice table with a nice caption.	11
---	---	----

List of Algorithms

1	The Colors from the Corporate Design of the University of Freiburg. The keywords are bold and the comments blue. This can be changed in <code>setup.tex</code>	12
---	--	----

Glossary

This document is incomplete. The external file associated with the glossary ‘main’ (which should be called `main.gls`) hasn’t been created.

Check the contents of the file `main.glo`. If it’s empty, that means you haven’t indexed any of your entries in this glossary (using commands like `\gls` or `\glsadd`) so this list can’t be generated. If the file isn’t empty, the document build process hasn’t been completed.

If you don’t want this glossary, add `nomain` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[nomain]{glossaries-extra}
```

Try one of the following:

- Add `automake` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[automake]{glossaries-extra}
```

- Run the external (Lua) application:

```
makeglossaries-lite.lua "main"
```

- Run the external (Perl) application:

```
makeglossaries "main"
```

Then rerun $\text{\LaTeX}$ on this document.

This message will be removed once the problem has been fixed.

References

- [1] Leander Bürkin. *LaTeX-Template*. Fachschaft TF Freiburg, 2025.
- [2] Yunhe Gao et al. Show and segment: Universal medical image segmentation via in-context learning. *arXiv preprint*, 2025.
- [3] others Hu. Medverse: A universal model for medical image segmentation. *arXiv preprint*, 2025.
- [4] Fabian Isensee, Paul F. Jaeger, Simon A. A. Kohl, Jens Petersen, and Klaus H. Maier-Hein. nnU-Net: A self-configuring method for deep learning-based biomedical image segmentation. *Nature Methods*, 2021.

Appendix